

IED INDIA PVT LTD

Empowering Future Leaders Through Excellence

Internship Management Portal

Full-Stack Web Application
Comprehensive Project Report

Project Title:	IED India Internship Management Portal
Developed By:	Mayuresh P
Technology:	Node.js, Express.js, MongoDB, Vanilla JS
Deployment:	Vercel (Serverless)
Version:	1.0.0
Date:	15 June 2026

Table of Contents

- 1. Abstract
 - 2. Introduction
 - 3. Problem Statement
 - 4. Objectives
 - 5. Technology Stack
 - 6. System Architecture
 - 7. Database Design
 - 8. Module Descriptions
 - 9. API Endpoints
 - 10. Security Implementation
 - 11. Deployment Architecture
 - 12. User Interface Design
 - 13. Testing & Quality Assurance
 - 14. Future Enhancements
 - 15. Conclusion
 - Appendix A — File Structure
 - Appendix B — Environment Variables
-

1. Abstract

The IED India Internship Management Portal is a full-stack web application developed to digitize and streamline the complete lifecycle of internship management at IED India Pvt Ltd. The system provides a centralized platform that handles every stage — from receiving and processing internship applications, scheduling and conducting interviews, onboarding selected candidates, managing daily attendance, assigning and tracking tasks, conducting performance evaluations, issuing digitally verifiable certificates, to facilitating internal communication through messaging and announcements.

Built using a modern technology stack comprising Node.js, Express.js, MongoDB, and vanilla JavaScript with a glassmorphic dark-mode UI, the portal implements Role-Based Access Control (RBAC) supporting four distinct user roles: Admin, HR, Mentor, and Intern. Each role is granted a tailored dashboard and permissions that ensure data security while maintaining operational efficiency. The application is deployed on Vercel's serverless infrastructure, leveraging MongoDB Atlas for cloud-hosted database persistence, and implements automated email notifications via Nodemailer for real-time status updates throughout the internship pipeline.

This report provides a comprehensive technical documentation of the system architecture, database schema design, individual module functionalities, RESTful API endpoints, security mechanisms, deployment strategy, and potential future enhancements for the portal.

2. Introduction

IED India Pvt Ltd is a dynamic organization that hosts a large number of interns across various departments including Digital Marketing, HR & Recruitment, Business Development, Social Media, Entrepreneurship Training, and IT Support. Managing internship operations across these departments involves a multitude of administrative tasks — processing applications, scheduling interviews, tracking attendance, assigning and reviewing tasks, evaluating performance, and generating certificates upon completion.

Prior to the development of this portal, these processes were managed manually using spreadsheets, emails, and physical records, leading to inefficiencies, data inconsistencies, delayed communication, and an overall suboptimal experience for both the administrative staff and the interns. The IED India Internship Management Portal was conceived to address these challenges by providing a single, unified, and automated platform for all stakeholders.

The application follows the Single Page Application (SPA) architecture pattern, where the client-side renders different modules dynamically without full page reloads, while the server-side exposes a comprehensive RESTful API. This separation of concerns ensures a clean, maintainable codebase and enables future scalability.

3. Problem Statement

Organizations that manage large internship programs face significant operational challenges when relying on traditional manual methods. The key problems identified at IED India include:

- **Fragmented Data Management:** Intern records, attendance logs, and evaluation data were scattered across multiple spreadsheets and email threads, making retrieval and analysis time-consuming and error-prone.
-

- **Inefficient Application Processing:** The absence of a centralized application portal meant that applications arrived via email, making it difficult to track, filter, and process them systematically.
- **Manual Interview Scheduling:** Coordinating interview schedules between interviewers and candidates required extensive back-and-forth communication via email or phone.
- **Lack of Real-Time Communication:** There was no internal messaging system, leading to reliance on personal messaging applications for work-related communication.
- **No Automated Notifications:** Status updates regarding application progress, interview schedules, and task assignments were communicated manually, leading to delays.
- **Absence of Digital Certificates:** Certificates were generated manually using word processors, with no mechanism for digital verification.
- **No Analytics or Reporting:** Management lacked a consolidated dashboard to view key metrics such as application trends, attendance patterns, and performance distributions.

The IED India Internship Management Portal addresses all of these challenges through automation, digitization, and centralization.

4. Objectives

The primary objectives of this project are:

- To develop a centralized web-based portal for managing all aspects of the internship lifecycle at IED India Pvt Ltd.
- To implement a Role-Based Access Control (RBAC) system with four distinct roles: Admin, HR, Mentor, and Intern, each with tailored permissions and dashboards.
- To automate the application intake process with a public-facing application form that captures personal information, internship preferences, and document uploads.
- To digitize the interview scheduling and feedback workflow with support for multiple interview modes (Zoom, Google Meet, Offline, Phone).
- To create a comprehensive attendance management system with check-in/check-out functionality, geo-location tracking, and multiple attendance types (Office, WFH, Leave).
- To build a task management module enabling mentors and admins to assign, track, and evaluate tasks with file attachments and feedback loops.
- To implement a structured weekly performance evaluation system with multi-dimensional rating criteria.
- To generate professionally designed PDF certificates with embedded QR codes for digital verification.
- To integrate automated email notifications for all critical events throughout the internship pipeline.
- To provide a real-time analytics dashboard with visualizations for management decision-making.
- To deploy the application on a serverless cloud platform (Vercel) for scalability and high availability.

5. Technology Stack

5.1 Backend Technologies

Technology	Version	Purpose
Node.js	v20+	JavaScript runtime for server-side execution
Express.js	4.19.2	Web framework for building RESTful APIs
MongoDB	7.x	NoSQL document database for data persistence
Mongoose	8.4.1	ODM for schema-based MongoDB modelling
JSON Web Token	9.0.2	Stateless authentication via signed tokens
bcryptjs	2.4.3	Password hashing (bcrypt, 12 salt rounds)
Nodemailer	6.9.14	Email delivery via SMTP / Resend API
PDFKit	0.15.0	Server-side PDF generation for certificates
QRCode	1.5.4	QR code generation for certificate verification
Multer	1.4.5	Multipart form-data handling for file uploads
Helmet	7.1.0	HTTP security headers middleware
Morgan	1.10.0	HTTP request logging middleware

5.2 Frontend Technologies

Technology	Version	Purpose
HTML5	-	Semantic markup and document structure
CSS3 (Custom)	-	Glassmorphism, gradients, dark-mode styling
Vanilla JavaScript	ES2020+	SPA routing, DOM manipulation, API calls
Lucide Icons	Latest	Modern SVG icon library
Chart.js	4.4.2	Interactive data visualizations and charts

5.3 Infrastructure & DevOps

Service	Purpose
Vercel	Serverless deployment with automatic CI/CD from GitHub
MongoDB Atlas	Cloud-hosted managed MongoDB cluster
GitHub	Source code version control and collaboration
Gmail SMTP / Resend	Transactional email delivery for notifications

6. System Architecture

The application follows a three-tier architecture comprising the Presentation Layer (client), the Application Layer (server/API), and the Data Layer (MongoDB database). The client communicates with the server exclusively through RESTful API calls, ensuring a clean separation of concerns.

6.1 Architectural Overview

- **Client Layer:** A Single Page Application (SPA) built with vanilla HTML, CSS, and JavaScript. The client dynamically renders 11 distinct modules based on the authenticated user's role. All API communication is handled through a centralized API helper class.
- **API Gateway:** The Express.js server acts as the API gateway, processing all incoming HTTP requests. It applies security middleware (Helmet, CORS), authentication (JWT), authorization (RBAC), and routes requests to the appropriate controller.
- **Business Logic Layer:** 13 controller files encapsulate the business logic for each module. Controllers interact with Mongoose models for data operations and invoke services (email, PDF generation, QR code) as needed.
- **Data Access Layer:** 12 Mongoose models define the schema structure and validation rules for MongoDB collections. The application uses MongoDB Atlas in production, with an in-memory MongoDB fallback for local development.
- **Service Layer:** Dedicated services handle cross-cutting concerns such as email delivery (with SMTP/Resend dual-provider support), PDF certificate generation (with in-memory buffer + MongoDB persistence for serverless compatibility), and QR code generation.

6.2 Request Lifecycle

A typical authenticated API request follows this lifecycle:

- 1. The client sends an HTTP request with the JWT token in the Authorization header.
- 2. The Express router matches the request to the appropriate route file.
- 3. The protect middleware extracts and verifies the JWT, attaching the authenticated user to the request object.
- 4. The authorize middleware checks if the user's role has permission to access the endpoint.
- 5. The controller executes the business logic, interacting with models and services as required.
- 6. The controller sends a JSON response back to the client with a success flag and relevant data.

7. Database Design

The application uses MongoDB, a document-oriented NoSQL database, with Mongoose as the Object Document Mapper (ODM). The database consists of 12 collections, each defined by a Mongoose schema with validation rules, default values, and indexing.

7.1 Collection Overview

Collection	Purpose	Key Fields
Users	All user accounts (admin, HR, mentor, intern)	name, email, password, role, department
Applications	Internship applications from public form	name, email, department, status, resume
Interviews	Interview scheduling and feedback	applicationId, scheduledAt, mode, result
Onboarding	Post-selection onboarding checklist	internId, offerLetterSent, documents
Attendance	Daily attendance records	internId, date, checkIn, checkOut, location
Tasks	Assigned work items with deadlines	title, assignedTo, deadline, priority, status
Messages	Direct messages between users	senderId, receiverId, content, read
Announcements	Organization-wide notices	title, content, targetRole, isPinned

Collection	Purpose	Key Fields
Evaluations	Weekly performance evaluations	internId, mentorId, week, ratings
Certificates	Generated completion certificates	internId, type, certNo, pdfPath, qrCode
Leaves	Leave requests from interns	internId, fromDate, toDate, reason, status
Uploads	Binary file storage (serverless)	filename, contentType, data (Buffer)

7.2 Key Design Decisions

- **Password Security:** User passwords are hashed using bcrypt with 12 salt rounds before storage. The password field is automatically excluded from query results via a custom toJSON method.
- **Referential Integrity:** Mongoose ObjectId references create relationships between collections (e.g., Application !' User, Task !' User). The populate() method is used for joins.
- **Compound Indexes:** The Attendance collection uses a compound unique index on {internId, date} to prevent duplicate entries per intern per day.
- **Binary File Storage:** The Upload collection stores file binary data as MongoDB Buffer fields, enabling file persistence on Vercel's read-only serverless filesystem.
- **Soft Deletes:** User accounts use an isActive boolean flag rather than physical deletion, preserving referential integrity and audit trails.

8. Module Descriptions

The portal consists of 11 functional modules, each serving a specific purpose in the internship management lifecycle.

8.1 Authentication & Authorization Module

Handles user registration, login, and session management. Implements JWT-based stateless authentication with token expiry. Supports role-based access control with four roles: Admin (full system access), HR (recruitment and onboarding), Mentor (task and evaluation management), and Intern (self-service portal). Includes a custom captcha verification on the login page.

8.2 Application Management Module

Provides a public-facing application form for prospective interns to submit their applications with personal details, internship preferences, document uploads (resume and photo), and a cover letter. The admin/HR dashboard displays all applications with search, filter, and pagination capabilities. Applications follow a defined status workflow: Applied !' Shortlisted !' Interview Scheduled !' Selected/Rejected/On Hold. Upon selection, an intern user account is automatically created with generated credentials, and an onboarding record is initiated.

8.3 Interview Management Module

Enables HR and Admin users to schedule interviews for shortlisted candidates. Supports multiple interview modes including Zoom, Google Meet, Offline, and Phone. Captures meeting links, interviewer assignment, date/time scheduling, and post-interview feedback with a scoring system (0-10). Interview results (Pass/Fail/Pending) directly influence the application status flow. Automated email notifications are sent to candidates with interview details.

8.4 Onboarding Module

Manages the post-selection onboarding process through a structured checklist. Tracks offer letter dispatch, orientation scheduling, document collection (resume, Aadhaar, college ID, photo), agreement upload, and welcome email dispatch. Each step can be individually toggled, and the overall onboarding status progresses from Pending !' In Progress !' Complete.

8.5 Attendance Management Module

Provides daily check-in and check-out functionality for interns. Supports multiple attendance types: Office, Work From Home (WFH), and Leave. Implements optional geo-location capture during check-in to verify the intern's physical presence. Features an attendance calendar view, monthly statistics, and admin/HR overviews with filtering by date range and intern. The system enforces a unique constraint per intern per day to prevent duplicate entries.

8.6 Task Management Module

Allows admins and mentors to create, assign, and track tasks for interns. Each task includes a title, description, deadline, priority level (Low/Medium/High/Urgent), and optional file attachments. Tasks follow a lifecycle: Todo !' In Progress !' Submitted !' Approved/Rejected/Revision. Interns can submit their work with notes and file uploads. Mentors provide feedback and completion scores (0-10). Overdue task detection is built into the UI.

8.7 Communication Module

Comprises two sub-modules: Direct Messaging and Announcements. The messaging system enables one-on-one communication between any users with text messages and file attachments. It features real-time message status tracking (sent/read), conversation threading by user pair, and unread message badges. The announcements system allows admins and HR to post organization-wide notices with optional role-targeting, department filtering, pin functionality, and expiry dates.

8.8 Performance Evaluation Module

Implements a structured weekly evaluation system where mentors rate interns across six dimensions: Communication, Teamwork, Leadership, Discipline, Technical Skills, and Task Completion, each on a 1-10 scale. The system auto-calculates an overall score and supports recommendation levels (Excellent/Good/Average/Needs Improvement). Evaluations include free-text fields for comments, strengths, and areas for improvement.

8.9 Leave Management Module

Enables interns to submit leave requests specifying date range, reason, and leave type (Sick, Casual, Personal, Emergency). The system auto-calculates the number of leave days. Admins and HR can approve or reject requests with optional remarks. Approved leaves automatically reflect in the attendance module.

8.10 Certificate Generation Module

Generates professionally designed PDF certificates with three types: Certificate of Completion, Letter of Recommendation, and Certificate of Appreciation. Each certificate features a deep navy background with gold accents, company branding, intern details, department, duration, performance rating, a unique certificate number, and an embedded QR code for verification. Certificates are generated in-memory using PDFKit and persisted to MongoDB for reliable access on Vercel's serverless platform. A public

verification endpoint allows anyone to verify a certificate by scanning its QR code.

8.11 Analytics Dashboard Module

Provides a real-time analytics dashboard for admins with key metrics including total interns, active applications, attendance rates, pending tasks, and more. Features interactive Chart.js visualizations including application status distribution (doughnut chart), department-wise intern distribution (bar chart), and attendance trends. The dashboard dynamically adapts based on the logged-in user's role, showing role-specific KPIs and quick-action buttons.

9. API Endpoints

The server exposes 13 route groups with over 50 individual API endpoints. All API endpoints follow RESTful conventions and return JSON responses with a consistent structure: { success: boolean, data/message }.

9.1 Authentication Routes (/api/auth)

Method	Endpoint	Auth	Description
POST	/api/auth/login	Public	Authenticate user, return JWT token
POST	/api/auth/register	Public	Register new user account
GET	/api/auth/me	Protected	Get current authenticated user profile
PUT	/api/auth/profile	Protected	Update user profile (name, photo, etc.)
PUT	/api/auth/password	Protected	Change password (requires current)

9.2 Application Routes (/api/applications)

Method	Endpoint	Auth	Description
GET	/api/applications	Protected	List all applications with filters
GET	/api/applications/:id	Protected	Get single application details
POST	/api/applications	Public	Submit new internship application
PATCH	/api/applications/:id/status	Admin/HR	Update application status
DELETE	/api/applications/:id	Admin	Delete an application

9.3 Interview Routes (/api/interviews)

Method	Endpoint	Auth	Description
GET	/api/interviews	Protected	List all scheduled interviews
POST	/api/interviews	Admin/HR	Schedule a new interview
PUT	/api/interviews/:id	Admin/HR	Update interview details
DELETE	/api/interviews/:id	Admin/HR	Cancel/delete an interview

9.4 Attendance Routes (/api/attendance)

Method	Endpoint	Auth	Description
GET	/api/attendance	Protected	Get attendance records (filtered)
POST	/api/attendance/checkin	Intern	Record daily check-in with location
PUT	/api/attendance/checkout	Intern	Record daily check-out

9.5 Task Routes (/api/tasks)

Method	Endpoint	Auth	Description
GET	/api/tasks	Protected	List tasks (role-filtered)
POST	/api/tasks	Admin/Mentor	Create and assign a new task
PUT	/api/tasks/:id	Admin/Mentor	Update task details
PATCH	/api/tasks/:id/submit	Intern	Submit task with notes/files
PATCH	/api/tasks/:id/review	Admin/Mentor	Review submitted task
DELETE	/api/tasks/:id	Admin	Delete a task

9.6 Other Route Groups

Route Group	Endpoints	Key Operations
/api/users	3	List, get, update, toggle active status
/api/onboarding	5	List, get, update steps, upload documents
/api/messages	4	List conversations, get/send messages
/api/announcements	4	List, create, update, delete announcements
/api/evaluations	4	List, create, update evaluations
/api/certificates	4	List, generate, download, verify
/api/leaves	3	List, apply, approve/reject leave requests
/api/analytics	1	Comprehensive dashboard statistics

10. Security Implementation

Security is a critical aspect of the portal, given that it handles personal information, authentication credentials, and sensitive organizational data.

10.1 Authentication Security

- **JWT-based Authentication:** Stateless authentication using JSON Web Tokens with configurable expiry (default 7 days). Tokens are stored client-side in localStorage and transmitted via the Authorization header.
- **Password Hashing:** All passwords are hashed using bcrypt with 12 salt rounds before database storage. The password field is excluded from all API responses via a custom toJSON transformation.

- Password Policy: Minimum 6-character password requirement enforced at the schema level.

10.2 Authorization Security

- Role-Based Access Control (RBAC): A centralized authorize() middleware restricts endpoint access based on user roles. Each route explicitly declares which roles are permitted.
- Resource-Level Filtering: Controllers apply role-based query filters (e.g., interns only see their own tasks, attendance, and evaluations).

10.3 Infrastructure Security

- Helmet.js: Automatically sets security-related HTTP headers including Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Strict-Transport-Security.
- CORS Configuration: Cross-Origin Resource Sharing is configured to allow only the designated client URL, preventing unauthorized cross-domain requests.
- Input Validation: Request data is validated at both the schema level (Mongoose validators) and the route level (express-validator).
- Rate Limiting: Vercel's built-in edge rate limiting provides DDoS protection at the infrastructure level.

11. Deployment Architecture

The application is deployed on Vercel, a serverless cloud platform that provides automatic deployments from the GitHub repository.

11.1 Serverless Adaptation

- Serverless Entry Point: The Express application is exported as a module and wrapped in a Vercel serverless function handler (api/index.js), enabling it to run within Vercel's serverless container.
- File System Workarounds: Since Vercel's serverless functions have a read-only filesystem (except /tmp), all file uploads are persisted to MongoDB via the Upload model rather than the local disk.
- Certificate Generation: PDF certificates are generated entirely in-memory using PDFKit buffers and saved to the MongoDB Upload collection, avoiding any filesystem dependency.
- Static Asset Routing: Vercel's rewrite rules in vercel.json route static assets (CSS, JS, images) directly to the client directory, while API and upload requests are forwarded to the serverless function.

11.2 Environment Configuration

Variable	Purpose
MONGODB_URI	MongoDB Atlas connection string
JWT_SECRET	Secret key for signing JWT tokens
EMAIL_USER	Gmail address for SMTP email delivery
EMAIL_PASS	Gmail App Password for SMTP authentication
CLIENT_URL	Frontend URL for CORS configuration
NODE_ENV	Environment flag (production)

12. User Interface Design

The user interface follows a modern dark-mode glassmorphic design language, prioritizing visual elegance, readability, and user experience.

12.1 Design System

- Color Palette: A navy-to-dark-blue gradient base with electric blue (#4f8ef7) as the primary accent and gold (#ffd700) for highlights and premium elements.
- Typography: System font stack (Inter, Segoe UI, sans-serif) with carefully calibrated sizes for hierarchy: 24px headings, 14px body, 12px labels.
- Glassmorphism: Cards and containers use semi-transparent backgrounds with backdrop-filter: blur() for a frosted-glass effect.
- Micro-Animations: Smooth CSS transitions on hover states, card scaling, button interactions, and page transitions.
- Responsive Design: Fully responsive layout with a collapsible sidebar on desktop and a bottom navigation bar on mobile devices.

12.2 Component Library

The frontend uses a custom-built CSS component library (components.css — 25,521 bytes) providing reusable components:

- Status Badges: Color-coded labels for dynamic states (applied, shortlisted, selected, approved, etc.).
- Data Tables: Styled tables with alternating row colors, hover effects, and responsive overflow handling.
- Modal Dialogs: Centered overlay modals with glassmorphic styling for forms, confirmations, and detail views.
- Toast Notifications: Animated slide-in notifications for success, error, and informational messages.
- Form Elements: Styled inputs, selects, textareas, and file uploaders with icon decorations and focus states.

13. Testing & Quality Assurance

13.1 Functional Testing

- All 50+ API endpoints were individually tested for correct response codes, data validation, and error handling.
- The complete application lifecycle was tested end-to-end: Application !' Interview !' Selection !' Onboarding !' Tasks !' Evaluation !' Certificate.
- Role-based access was verified to ensure unauthorized users cannot access restricted endpoints or UI modules.

13.2 Integration Testing

- Database operations were tested with MongoDB Atlas and the in-memory MongoDB fallback to ensure cross-environment compatibility.
 - Email notifications were verified on the production Vercel environment to confirm SMTP connectivity and delivery.
-

- File uploads were tested on both local (filesystem) and serverless (MongoDB binary) storage backends.

13.3 Cross-Browser & Responsive Testing

- Tested on Chrome, Firefox, Edge, and Safari for rendering consistency.
- Responsive design validated across desktop (1920px), tablet (768px), and mobile (375px) viewports.
- Touch interactions verified on mobile devices for the bottom navigation bar and swipe gestures.

14. Future Enhancements

While the current version provides comprehensive internship management capabilities, the following enhancements are planned:

- Real-Time Notifications: Implement WebSocket (Socket.io) connections for instant push notifications and live messaging without page refresh.
- Advanced Analytics: Add predictive analytics using historical data to forecast intern performance, dropout risk, and optimal department allocation.
- Custom Domain Email: Migrate from Gmail SMTP to a verified custom domain email provider (e.g., Resend with iedindia.com) to ensure emails land in the primary inbox.
- Mobile Application: Develop native iOS and Android applications using React Native for improved mobile experience, especially for attendance check-in with biometric authentication.
- Video Interview Integration: Build native video conferencing capabilities directly into the portal.
- Automated Task Grading: Integrate AI-powered code review and automated grading for technical tasks submitted by IT department interns.
- Multi-Language Support: Add internationalization (i18n) to support Hindi and other regional languages for broader accessibility.
- Audit Logging: Implement comprehensive audit logs tracking all administrative actions for compliance and accountability.

15. Conclusion

The IED India Internship Management Portal successfully addresses the operational challenges of managing a large-scale internship program. By digitizing and automating the entire internship lifecycle — from application intake through certificate issuance — the system significantly reduces administrative overhead, eliminates data fragmentation, improves communication efficiency, and enhances the overall experience for both staff and interns.

The technology choices — Node.js, Express.js, MongoDB, and vanilla JavaScript — provide a robust, maintainable, and performant foundation. The serverless deployment on Vercel ensures high availability, automatic scaling, and zero-maintenance infrastructure. The modular architecture of the codebase, with clear separation between routes, controllers, models, and services, enables straightforward future extension and maintenance.

With approximately 10,600+ lines of source code across 72 files, the project demonstrates the successful implementation of a production-grade, full-stack web application that solves real-world organizational challenges through modern software engineering practices.

Appendix A — Project File Structure

```
IED INDIA INTERNSHIP PORTAL/
% % % client/
% % % % % css/
% % % % % % % main.css (16,503 bytes)
% % % % % % % components.css (25,521 bytes)
% % % % % % % sidebar.css ( 9,373 bytes)
% % % % % % % dashboard.css ( 7,967 bytes)
% % % % % % % js/
% % % % % % % api.js ( 5,272 bytes)
% % % % % % % auth.js ( 8,605 bytes)
% % % % % % % app.js (27,298 bytes)
% % % % % % % modules/
% % % % % % % dashboard.js (19,198 bytes)
% % % % % % % applications.js (15,371 bytes)
% % % % % % % interviews.js (19,003 bytes)
% % % % % % % onboarding.js (16,360 bytes)
% % % % % % % attendance.js (13,548 bytes)
% % % % % % % tasks.js (11,826 bytes)
% % % % % % % communication.js (19,186 bytes)
% % % % % % % evaluation.js (16,368 bytes)
% % % % % % % certificates.js ( 8,326 bytes)
% % % % % % % analytics.js ( 7,581 bytes)
% % % % % % % users.js (12,209 bytes)
% % % % % % % index.html (16,153 bytes)
% % % server/
% % % % % config/db.js
% % % % % % % controllers/ (13 files)
% % % % % % % middleware/ ( 3 files)
% % % % % % % models/ (12 files)
% % % % % % % routes/ (13 files)
% % % % % % % services/ ( 3 files)
% % % % % % % server.js ( 3,947 bytes)
% % % % % % % api/index.js (Vercel serverless entry)
% % % % % % % vercel.json
% % % % % % % package.json
% % % % % % % README.md
```

Appendix B — Environment Variables

The following environment variables must be configured for the application to function correctly in production:

Variable	Required	Description
MONGODB_URI	Yes	MongoDB Atlas connection URI string
JWT_SECRET	Yes	Secret key for JWT token signing
EMAIL_USER	Yes*	Gmail address for SMTP email delivery
EMAIL_PASS	Yes*	Gmail App Password (16-char token)
RESEND_API_KEY	Alt*	Resend.com API key (alternative to SMTP)
EMAIL_FROM	No	Custom "From" address for emails
CLIENT_URL	No	Frontend URL for CORS (defaults to *)
NODE_ENV	No	Environment mode (development/production)
PORT	No	Server port (defaults to 5000)

* Either EMAIL_USER + EMAIL_PASS or RESEND_API_KEY must be configured for email functionality.

This report was generated on 15 June 2026 at 11:04 am.

© 2026 IED India Pvt Ltd. All rights reserved.
